

Intelligent Vehicles Exercise 2 Report:

Dead Reckoning Estimation



Group 2, Binay Kumar Shah & Kabir Tamari
D Intelligent Vehicles
February 11, 2026

1 Odometry with a differential drive robot

1.1 The kinematic model with the uncertainty parameters for the forward direction and heading

Task 1: Calculate the changes in the robot's forward direction (Δd) and heading ($\Delta\theta$), i.e., the changes expressed in the robot coordinate system caused by the latest movement. Also, calculate the variances (covariances) of (Δd) and ($\Delta\theta$). What uncertainties do you assume?

- **Change in Forward Distance and Heading:** These are calculated by the given equation:

$$\Delta d = \frac{\Delta s_r + \Delta s_l}{2}$$

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{B}$$

where,

- Δs_r and Δs_l are the right and left wheel displacements.
- B is the Wheel base .

- **Covariance Calculation:**

$$C_u = \begin{bmatrix} \frac{\sigma_r^2 + \sigma_l^2}{4} & \frac{\sigma_r^2 - \sigma_l^2}{2B} \\ \frac{\sigma_r^2 - \sigma_l^2}{2B} & \frac{\sigma_r^2 + \sigma_l^2}{B^2} \end{bmatrix}$$

- **Assumed Uncertainties:**

- We assume that the left and right wheel encoder errors are independent and have equal variance i.e., $\sigma_r = \sigma_l$. Under this assumption, the cross-covariance term between Δd and $\Delta\theta$ becomes zero.
- We also assumed that wheel encoder has an error of 0.5 mm for every 12 mm traveled.
- The robot starts with a known heading of 90° and zero initial position uncertainty

Task 2: Calculate the new state variables of the robot under the assumption that the robot moves according to a circular trajectory (at this point, you could skip the compensation term given in Wang 1988 [1]). At the same time, calculate the covariance matrix of these new positions. You can assume that the robot always starts at the origin with a heading of 90° , i.e., $(x_0, y_0, \theta_0) = (0, 0, 90\pi/180)$.

- **State Variables:** The state vector S is defined by the local coordinates (x, y) and it's orientation(θ) i.e. $S = [x, y, \theta]^T$. With (Δd_r) and (Δd_l) as the displacement of the right and left wheel, we first find the relative movement:

1. Average Distance:

$$\Delta D = \frac{\Delta d_r + \Delta d_l}{2}$$

2. Change in Heading:

$$\Delta\theta = \frac{\Delta d_r - \Delta d_l}{b}$$

where b is the wheelbase.

Now, new state variable at time k are calculated from the previous state $k - 1$ as :

$$x_k = x_{k-1} + \Delta D \cos\left(\theta_{k-1} + \frac{\Delta\theta}{2}\right)$$

$$y_k = y_{k-1} + \Delta D \sin\left(\theta_{k-1} + \frac{\Delta\theta}{2}\right)$$

$$\theta_k = \theta_{k-1} + \Delta\theta$$

- **Covariance Propagation:** The update uncertainty of the robot's position is given as:

$$\Sigma_{X_k} = J_{X_{k-1}} \Sigma_{X_{k-1}} J_{X_{k-1}}^T + J_{\Delta d \Delta \theta} \Sigma_{\Delta d \Delta \theta} J_{\Delta d \Delta \theta}^T \quad (2) \text{ [cite: 226]}$$

where,

1. X_{k-1} is the previous uncertainty
2. $\Sigma_{\Delta d \Delta \theta}$ is the covariance matrix of the input noise (encoder errors)

$$\Sigma_{\Delta d \Delta \theta} = \begin{bmatrix} \sigma_{\Delta d}^2 & 0 \\ 0 & \sigma_{\Delta \theta}^2 \end{bmatrix}$$

3. $\nabla J_{X_{k-1}}$ is the Jacobian with respect to the state and is given by :

$$J_{X_{k-1}} = \begin{bmatrix} 1 & 0 & -\Delta d \sin(\theta_{k-1} + \frac{\Delta \theta}{2}) \\ 0 & 1 & \Delta d \cos(\theta_{k-1} + \frac{\Delta \theta}{2}) \\ 0 & 0 & 1 \end{bmatrix}$$

4. $J_{\Delta d \Delta \theta}$ is the Jacobian with respect to the uncertainty parameters:

$$J_{\Delta d \Delta \theta} = \begin{bmatrix} \cos(\theta_{k-1} + \frac{\Delta \theta}{2}) & -\frac{\Delta d}{2} \sin(\theta_{k-1} + \frac{\Delta \theta}{2}) \\ \sin(\theta_{k-1} + \frac{\Delta \theta}{2}) & \frac{\Delta d}{2} \cos(\theta_{k-1} + \frac{\Delta \theta}{2}) \\ 0 & 1 \end{bmatrix}$$

- **Result:** Figure 1 shows the output of the robot trajectory with no compensation applied.

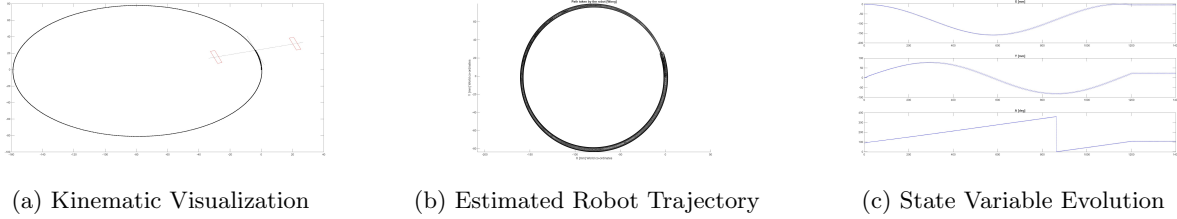


Figure 1: Robot path and state evolution with no compensation applied

Task 3: Run the entire sequence of the encoder, but also incorporate the compensation term given in the paper by Wang [1]. Are the estimated state variables and covariance matrices the same? If you read the encoder values less often, i.e., if you read them 2, 5, or maybe ten times as seldom or more, what will happen to the state variables? When do the state variables start to differ – and why? (Hint: Check, e.g., the Δd and $\Delta \theta$ values).

- **Comparison with Wang's Compensation Term:** Figure 2 shows the output of the robot trajectory on applying compensation term $\frac{\sin(\frac{\Delta \theta}{2})}{\frac{\Delta \theta}{2}}$ at sampling rate 1 : 1 and it shows no visible differences from that of robot trajectory without compensation 1.

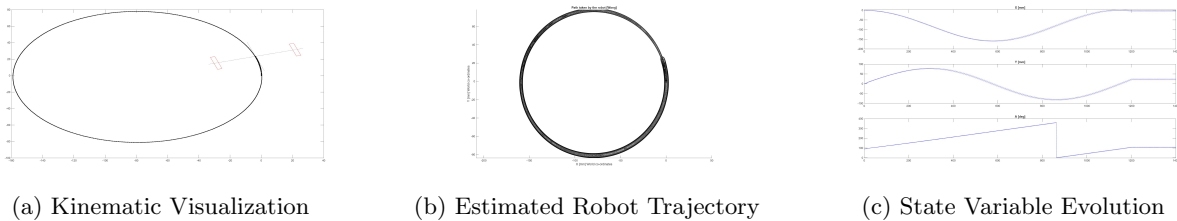
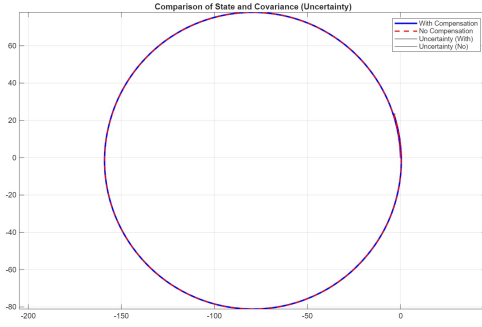


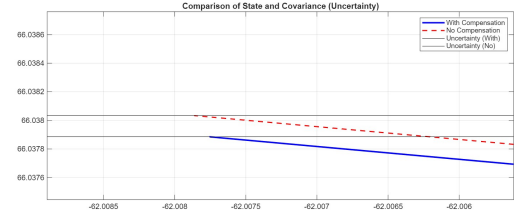
Figure 2: Robot path and state evolution with compensation applied

- **State Variables and Covariance Matrices:** On comparing their trajectories (with and without compensation) at 1 : 1 sampling rate and zooming the particular section we can see small difference between the model trajectory and it increase towards the end of the path which was found to be **0.000051 mm** shown in figure 3. The difference is very small, this is because the rate of angular change, $\Delta\theta$, becomes very small i.e., $\Delta\theta$ approaches to 0 which intends the compensation factor approaches 1. This state variable difference gets bigger and easily visible as we reduce the sample rate, for instance, we perform at sample rate 1 : 10 shown in figure 4 and their model trajectories difference at end was 0.014730mm

However, the covariance matrices were found to be the same. This is because the covariance propagation is calculated using the Jacobian matrices, which depend on Δd and $\Delta\theta$. Since the compensation term only modifies the position update (scaling Δd) and does not change $\Delta\theta$ or the Jacobian structure, the covariance propagation equations remain identical. It was also observed that covariance values decrease when encoder values are sampled less frequently, because fewer update steps mean uncertainty accumulates fewer times.

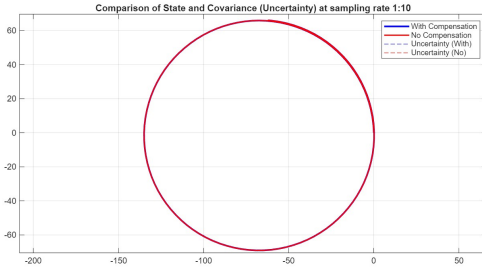


(a) Full trajectory at 1:1 sampling.

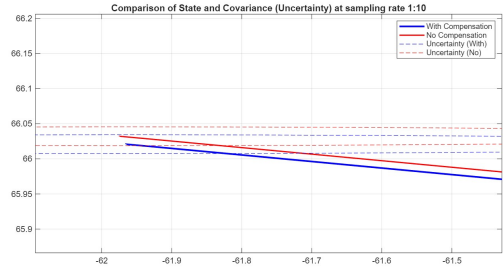


(b) Zoomed view of the end-point (1:1).

Figure 3: Odometry performance at a 1:1 sampling rate. At this high resolution, the inward drift is negligible, and both models' state variables and uncertainty ellipses overlap significantly.



(a) Full trajectory at 1:10 sampling.

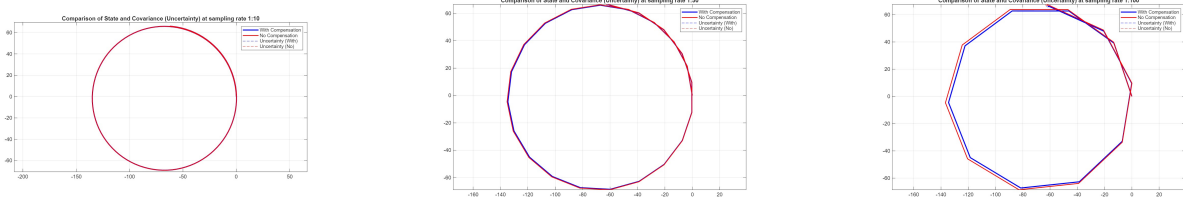


(b) Zoomed view of the end-point (1:10).

Figure 4: Odometry performance at a 1:10 sampling rate. The uncompensated model (red) drifts inward due to the straight-line (chord) approximation. The separation of the uncertainty ellipses confirms that the state variables (x, y) have diverged.

- **Effects of Reduced Sampling Rates:** Figure 5 shows the impact on state variables on changing

sampling rate from 10 , 50 to 100 respectively. Here we found that the state variable (x,y) diverges because the discrete motion model fails to capture the continuous curvature at low sampling frequencies. From the comparison we can say that as the sample interval increases, the circular path bFrom the comparison, it is concluded that as the sample interval increases, the circular path begins to show chord approximations, and for the angles, updates are much sharper because each step covers more ground, leading to a visible divergence in the state over time, this causes a transition from a circular to a polygonal trajectory.



(a) Full trajectory at 1:10 sampling.

(b) Full trajectory at 1:50 sampling.

(c) Full trajectory at 1:100 sampling.

Figure 5: Odometry performance at different sampling rate.

- **State variable differ:** The state variables start to differ significantly when the change in heading $\Delta\theta$ per sample becomes large. This happens because the difference between the straight chord and the actual arc grows, as $\Delta\theta$ increases. This causes the wang compensation term $\frac{\sin(\Delta\theta/2)}{\Delta\theta/2}$ to stretch the straight line back into the correct arc length.

Task 4: How are the covariance matrices evolving during the run (plot the entire run together with the calculated covariance matrices)? You don't have the actual values of the state variables (a common problem in the mobile robot community) – but are the uncertainties realistic, nonetheless?

In order to visualize the covariance matrices evolving more clearly, plotting was done after 50th step as shown by the code below and hence we got a plot as shown in figure 6. The result clearly shows that the ellipses get larger and more elongated as the robot moves during the run. This highlights that the model is realistic despite not having an actual value of state variable because in dead-reckoning, error are cumulative and once an error occurs, every subsequent calculations uncertainty grows as well.

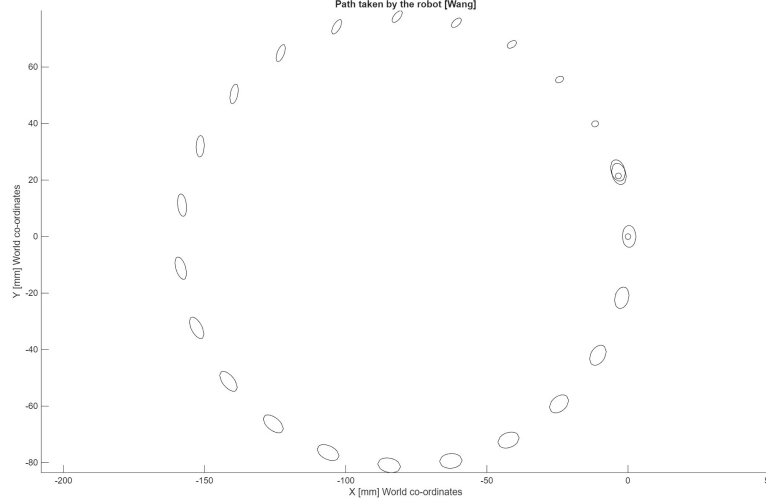


Figure 6: Evolving of Covariance matrices

```

....
hold on;
% plotting the covariance after 50th step
for kk = 1:50:N,
    C = [P(kk,1:3);P(kk,4:6);P(kk,7:9)];
    plot_uncertainty([X(kk) Y(kk) A(kk)]', C, 1, 2);
end;
hold off;
....

```

1.2 The kinematic model (different from Task 1-4) with the uncertainty parameters, distance driven by the left and right wheel

Task 5: Once more, run the same test but use less well-known wheel diameter and wheelbase values, e.g., $D = 14\text{mm}$ and $WB = 45\text{mm}$. How does it affect the estimated state variables? Plot the trajectories in the same plot as the other trajectories. What assumptions about these errors would you make?

We performed the dead-reckoning simulation with modified parameters: Wheel Diameter $D = 14\text{ mm}$ (vs. standard 15.3 mm) and Wheelbase $WB = 45\text{ mm}$ (vs. standard 53 mm). Previously, the uncertainty parameters were the forward displacement Δd and the heading change $\Delta\theta$. However, for Tasks 5–6 we use a different kinematic model where the uncertainty parameters are the distances driven by the right and left wheels, ΔR and ΔL , together with the wheelbase b . The state update equation (Equation 8) now becomes:

$$X_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} x_{k-1} + \frac{\Delta R + \Delta L}{2} \cos\left(\theta_{k-1} + \frac{\Delta R - \Delta L}{2b}\right) \\ y_{k-1} + \frac{\Delta R + \Delta L}{2} \sin\left(\theta_{k-1} + \frac{\Delta R - \Delta L}{2b}\right) \\ \theta_{k-1} + \frac{\Delta R - \Delta L}{b} \end{bmatrix} \quad (1)$$

where ΔR and ΔL are the incremental distances of the right and left wheel, and b is the wheelbase. Here we use the substitutions $\Delta d = \frac{\Delta R + \Delta L}{2}$ and $\Delta\theta = \frac{\Delta R - \Delta L}{b}$.

Since the uncertainties now originate from ΔR , ΔL , and the wheelbase b , the covariance propagation

equation (Equation 9) includes their contributions:

$$\Sigma_{X_k} = J_{X_{k-1}} \Sigma_{X_{k-1}} J_{X_{k-1}}^T + J_{\Delta R \Delta L} \Sigma_{\Delta R \Delta L} J_{\Delta R \Delta L}^T + J_b \Sigma_b J_b^T \quad (2)$$

where $J_{X_{k-1}}$ is the Jacobian with respect to the state variables (same form as Equation 3). The Jacobian with respect to the wheel displacements $(\Delta R, \Delta L)$ is:

$$J_{\Delta R \Delta L} = \begin{bmatrix} \frac{1}{2} \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) - \frac{\Delta d}{2b} \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) & \frac{1}{2} \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) + \frac{\Delta d}{2b} \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ \frac{1}{2} \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) + \frac{\Delta d}{2b} \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) & \frac{1}{2} \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) - \frac{\Delta d}{2b} \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ \frac{1}{b} & -\frac{1}{b} \end{bmatrix} \quad (3)$$

The Jacobian with respect to the wheelbase b is:

$$J_b = \begin{bmatrix} \frac{\Delta d(\Delta R - \Delta L)}{2b^2} \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ -\frac{\Delta d(\Delta R - \Delta L)}{2b^2} \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ -\frac{\Delta R - \Delta L}{b^2} \end{bmatrix} \quad (4)$$

The input covariance matrix is:

$$\Sigma_{\Delta R \Delta L} = \begin{bmatrix} \sigma_R^2 & 0 \\ 0 & \sigma_L^2 \end{bmatrix} \quad (5)$$

where σ_R^2 and σ_L^2 are the variances of the right and left wheel displacements (same encoder uncertainty as before). The wheelbase uncertainty is $\Sigma_b = \sigma_b^2$. We use the same encoder uncertainty as in previous tasks. The additional term $J_b \Sigma_b J_b^T$ captures the effect of uncertainty in the wheelbase parameter on the estimated robot pose.

1.2.1 Findings:

Then the estimated trajectory differs significantly from the standard one, as shown in the comparison plot 7.

1. **Effect of Reduced Wheel Diameter (D):** The incremental distance traveled Δs is proportional to D .

$$\Delta s \propto D$$

Using $D = 14$ mm instead of 15.3 mm causes an underestimation of the distance traveled by a factor of $\frac{14}{15.3} \approx 0.915$. Consequently, the total length of the estimated path is approximately 8.5% shorter than the standard path.

2. **Effect of Reduced Wheelbase (WB):** The change in orientation $\Delta\theta$ is inversely proportional to WB .

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{WB}$$

Using $WB = 45$ mm instead of 53 mm causes an overestimation of the turning angle by a factor of $\frac{53}{45} \approx 1.18$. This means the estimated robot orientation changes more rapidly than the standard estimation. For a circular path, this results in a tighter predicted turning radius, causing the trajectory to spiral inward or form a smaller circle.

Assumption: We can assume that the diameter error acts as a linear scaling factor on the path length, and the wheelbase error acts as a scaling factor on curvature. This is particularly critical because orientation errors integrate into position errors, causing the estimated path to diverge rapidly from the true path (accumulation of error).

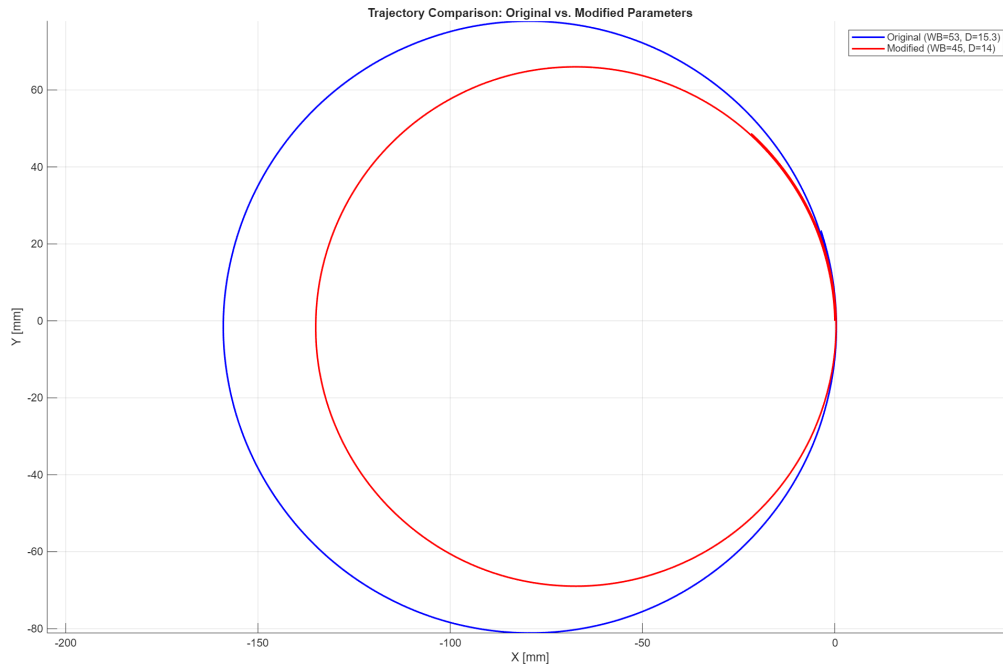


Figure 7: Standard Wheelbase vs Modified Wheel Base

Task 6: Repeat the experiment in task 5, but use the data file 'khepera.txt'. Plot the trajectories in the same plot as the other trajectories, i.e., different wheelbase and wheel diameter!

Figure 8 shows the comparison plotting for 'khepera.txt' file. We can observe that, the drift started shortly after the starting motion, where small errors in θ are the leading cause of position drift.

Moreover, we also can observed that the error are not random in that case it would have been wiggling around, rather found to be a systematic error because the red path over the time follows a consistently different pattern, it proves model is misinterpreting the physical reality of the robot's motion.

At last the error, the covariance matrices grows and elongated over time, demonstrating that the model in dead-reckoning.

This all highlight one important rules in odometry that the precise calibration is mandatory.

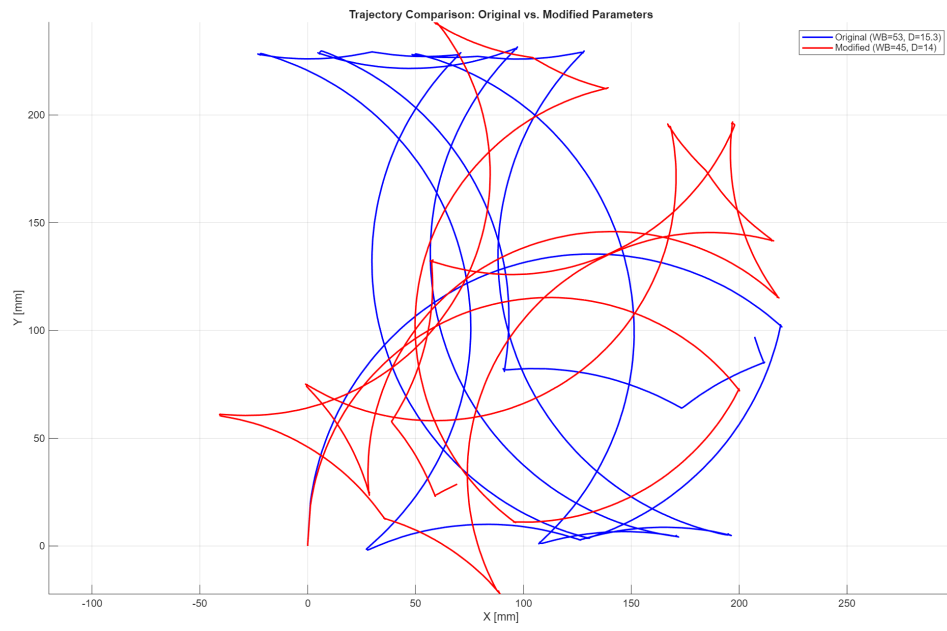


Figure 8: Trajectories comparison between original and modified parameter for khepera.txt

2 Dead-reckoning using a Three-Wheeled vehicle

Task 7: Calculate the new state variables of the robot. Also, calculate the covariance matrix of these new positions. (To do this, you have to derive the Jacobian matrices w.r.t. the uncertain parameters. Follow the example given in the Wang paper [1] or chapter 5 in the textbook [2].) The robot always starts in the first ground truth position and with the first ground truth heading, i.e., $(x_0, y_0, \vartheta_0) = (\text{TrueX}(0), \text{TrueY}(0), \text{True}\vartheta(0))$.

The Snowwhite robot uses a steer-drive configuration with one steerable front wheel and two passive rear wheels. The control inputs are the speed v and steering angle α of the front wheel, sampled at period $T = 0.050$ s. The wheelbase is $L = 680$ mm. To predict robot's motion we use motion model that calculates the relative displacements per time step (Equations 11–12):

$$\Delta d = v \cos(\alpha) \cdot T \quad (6)$$

$$\Delta \theta = \frac{v \sin(\alpha) \cdot T}{L} \quad (7)$$

where v is the speed, α is the steering angle and T is the sampling time which is also the new uncertainty parameters. The new state update model (Equation 13) becomes:

$$\mathbf{X}_k = \begin{bmatrix} x_{k-1} + v \cos(\alpha) T \cos\left(\theta_{k-1} + \frac{v \sin(\alpha) T}{2L}\right) \\ y_{k-1} + v \cos(\alpha) T \sin\left(\theta_{k-1} + \frac{v \sin(\alpha) T}{2L}\right) \\ \theta_{k-1} + \frac{v \sin(\alpha) T}{L} \end{bmatrix} \quad (8)$$

Using the law of error propagation we can calculate the covariance of the new state using (Equation 14):

$$\Sigma_{\mathbf{X}_k} = J_{\mathbf{X}_{k-1}} \Sigma_{\mathbf{X}_{k-1}} J_{\mathbf{X}_{k-1}}^T + J_{v\alpha T} \Sigma_{v\alpha T} J_{v\alpha T}^T \quad (9)$$

The Jacobian w.r.t. the state $J_{\mathbf{X}_{k-1}}$ has the same form as before. The Jacobian w.r.t. the input parameters (v, α, T) is a 3×3 matrix derived by differentiating our motion model equation above.. Letting $\varphi = \theta_{k-1} + \frac{v \sin(\alpha) T}{2L}$:

$$J_{v\alpha T} = \begin{bmatrix} \frac{\partial f_1}{\partial v} & \frac{\partial f_1}{\partial \alpha} & \frac{\partial f_1}{\partial T} \\ \frac{\partial f_2}{\partial v} & \frac{\partial f_2}{\partial \alpha} & \frac{\partial f_2}{\partial T} \\ \frac{\partial f_3}{\partial v} & \frac{\partial f_3}{\partial \alpha} & \frac{\partial f_3}{\partial T} \end{bmatrix} \quad (10)$$

where the individual partial derivatives are:

$$\frac{\partial f_1}{\partial v} = T \cos \alpha \cos \varphi - \frac{v T^2 \cos \alpha \sin \alpha}{2L} \sin \varphi \quad (11)$$

$$\frac{\partial f_1}{\partial \alpha} = -v T \sin \alpha \cos \varphi - \frac{v^2 T^2 \cos^2 \alpha}{2L} \sin \varphi \quad (12)$$

$$\frac{\partial f_1}{\partial T} = v \cos \alpha \cos \varphi - \frac{v^2 T \cos \alpha \sin \alpha}{2L} \sin \varphi \quad (13)$$

$$\frac{\partial f_2}{\partial v} = T \cos \alpha \sin \varphi + \frac{v T^2 \cos \alpha \sin \alpha}{2L} \cos \varphi \quad (14)$$

$$\frac{\partial f_2}{\partial \alpha} = -v T \sin \alpha \sin \varphi + \frac{v^2 T^2 \cos^2 \alpha}{2L} \cos \varphi \quad (15)$$

$$\frac{\partial f_2}{\partial T} = v \cos \alpha \sin \varphi + \frac{v^2 T \cos \alpha \sin \alpha}{2L} \cos \varphi \quad (16)$$

$$\frac{\partial f_3}{\partial v} = \frac{T \sin \alpha}{L}, \quad \frac{\partial f_3}{\partial \alpha} = \frac{v T \cos \alpha}{L}, \quad \frac{\partial f_3}{\partial T} = \frac{v \sin \alpha}{L} \quad (17)$$

And finally, the input covariance matrix for our uncertainty input is given by the follow:

$$\Sigma_{v\alpha T} = \begin{bmatrix} \sigma_v^2 & 0 & 0 \\ 0 & \sigma_\alpha^2 & 0 \\ 0 & 0 & \sigma_T^2 \end{bmatrix} \quad (18)$$

For the assumed uncertainties, we model the speed noise as a proportion of the current speed, similar to how the odometry encoder error was modeled as a fraction of the distance traveled in Tasks 5–6. Specifically:

- **Speed noise:** $\sigma_v = 0.05 \cdot |v|$, i.e. 5% of the current speed. This is consistent with the proportional error assumption used in the odometry case, where higher speeds naturally lead to larger absolute uncertainty due to wheel slip and sensor noise.
- **Steering angle noise:** $\sigma_\alpha = 1^\circ = \pi/180$ rad, reflecting mechanical play in the steering mechanism.
- **Sampling time jitter:** $\sigma_T = 0.001$ s, which is small relative to $T = 0.050$ s but included for completeness.

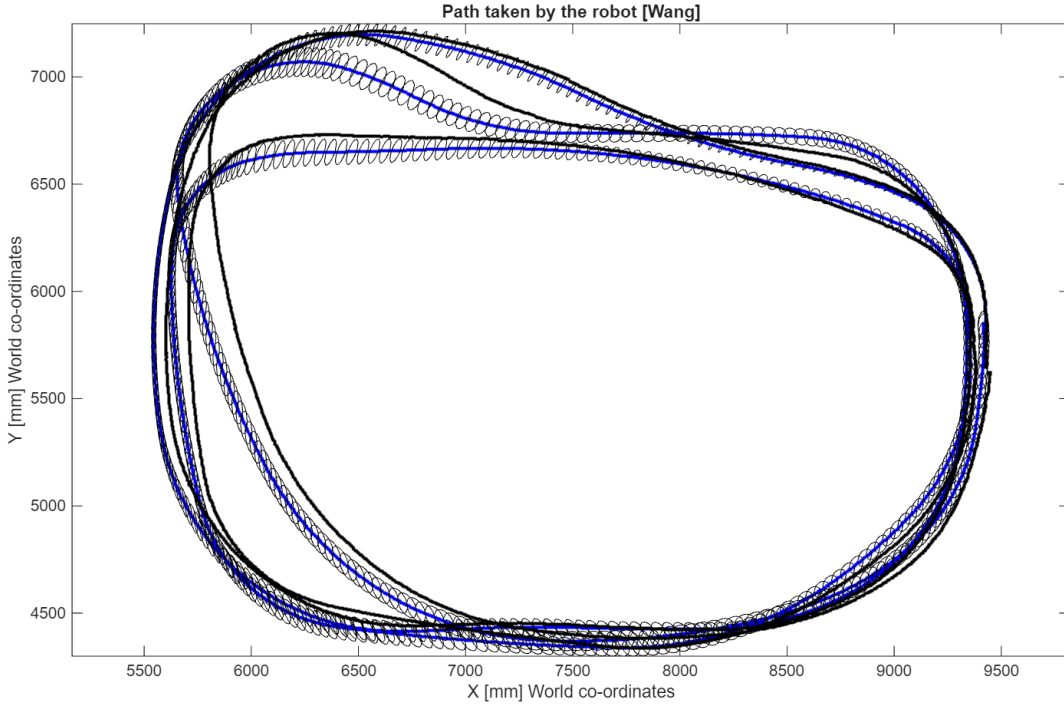


Figure 9: Dead-reckoning trajectory (blue) with error ellipses compared against the ground truth (black) for the Snowwhite robot.

In Figure 9, the blue estimated path initially overlaps with the black ground truth but gradually drifts away. This happens because small errors in speed and steering angle accumulate at every step. Once the heading θ drifts from the true value, all subsequent position updates are pushed in a slightly wrong direction, causing the error to compound. The error ellipses visibly grow along the path, reflecting this increasing uncertainty. The overall path shape is still recognizable confirming the kinematic model is correct and it is the accumulated noise that causes the divergence.

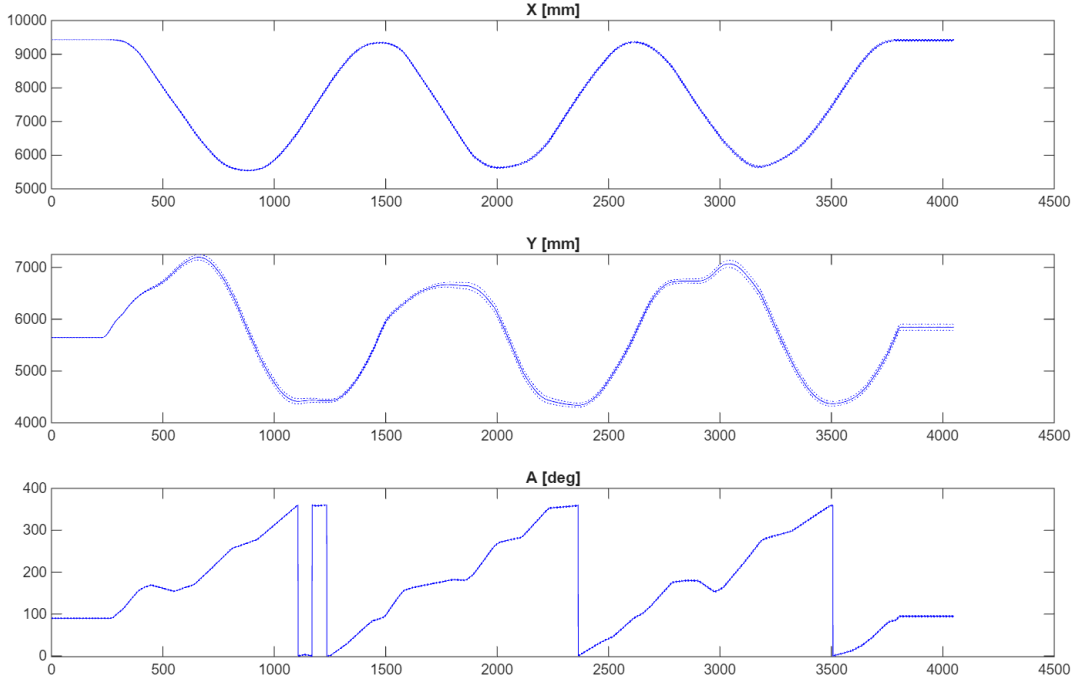


Figure 10: Estimated state variables (X, Y, θ) over the full run with $\pm 1\sigma$ uncertainty bounds (dotted lines).

Figure 10 shows X, Y , and heading A over all time steps, with dotted $\pm 1\sigma$ bounds from the covariance diagonal. The uncertainty envelope starts tight and widens steadily as the run progresses. The heading plot shows 360° wraparound jumps where the angle wraps past 2π . Without external corrections, the predicted uncertainty grows continuously at every step.

Task 8: Run the sequence and compare estimated state variables to ground truth. Also, compare the error in the state variable estimates to the standard deviations of the estimated variances. What errors do you assume in the steering angle and the speed?

To understand the effect of noise we ran the full sequence and computed the estimation error at each time step as the difference between our dead-reckoning estimate and the ground truth:

$$e_x(k) = X(k) - \text{TrueX}(k), \quad e_y(k) = Y(k) - \text{TrueY}(k), \quad e_\theta(k) = \theta(k) - \text{True}\theta(k) \quad (19)$$

We then compared these errors against the predicted standard deviations $\sigma_x(k) = \sqrt{P_{11}(k)}$, $\sigma_y(k) = \sqrt{P_{22}(k)}$, and $\sigma_\theta(k) = \sqrt{P_{33}(k)}$ from our covariance matrix.

To find the right noise parameters and experiment a bit we start with a low estimate, try a high estimate, then settle on values in between where the $\pm 1\sigma$ bounds track the actual errors.

We started with conservative noise values:

Parameter	Value
σ_v	$0.05 \cdot v $
σ_α	1°
σ_T	0.001 s

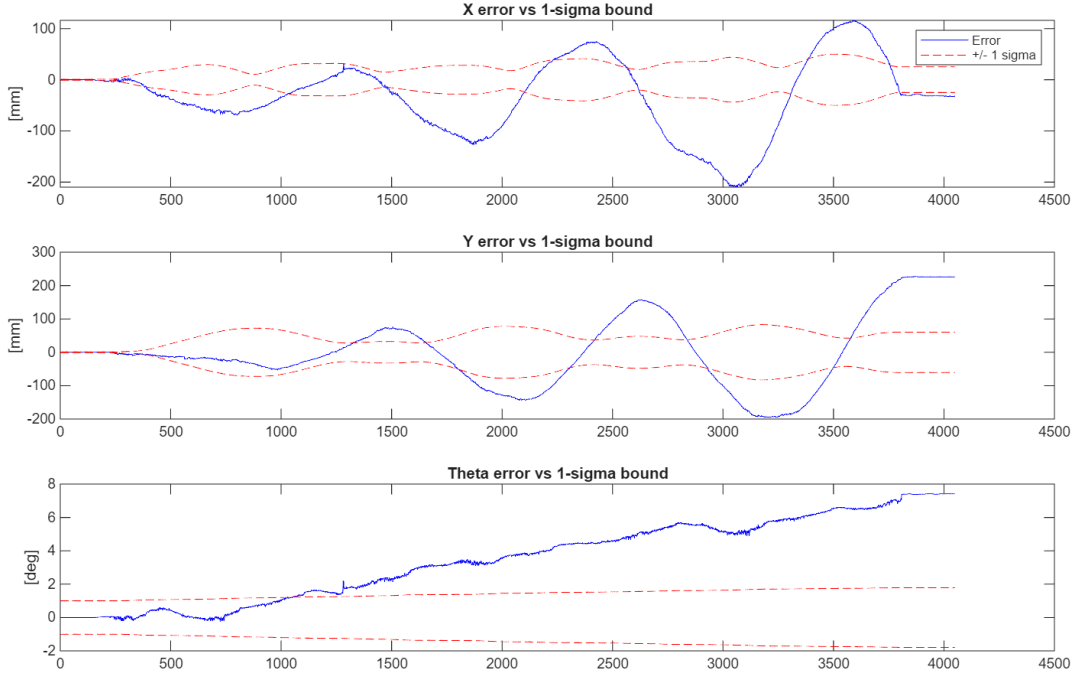


Figure 11: Estimation error (blue) vs predicted $\pm 1\sigma$ bounds (red dashed) with low noise parameters.

As seen in Figure 11, the X and Y errors exceed the bounds in the second half of the run so does the heading angle. The heading error reaches $7-8^\circ$ while the predicted σ_θ only reaches $\sim 2^\circ$ which shows we really underestimate.

High noise ($10\times$). We then increased all noise parameters by a factor of 10 ($\sigma_v = 0.50|v|$, $\sigma_\alpha = 10^\circ$, $\sigma_T = 0.01$ s).

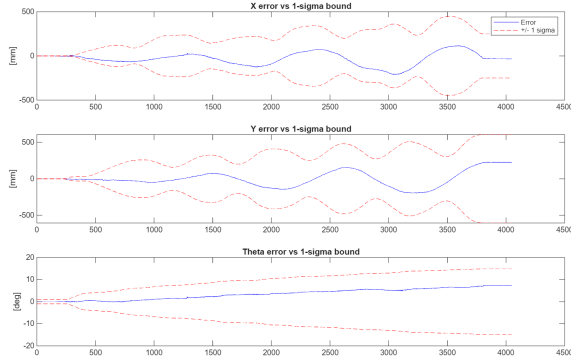


Figure 12: Estimation error (blue) vs $\pm 1\sigma$ bounds (red dashed) with $10\times$ noise parameters.

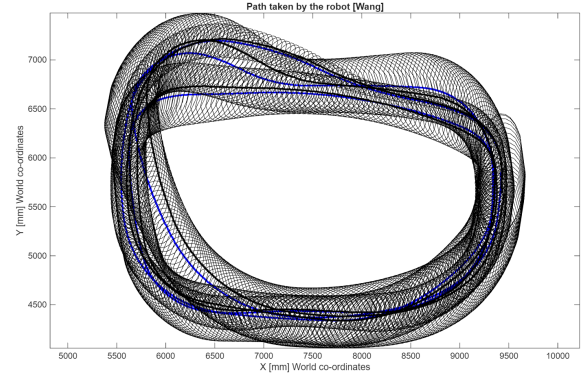


Figure 13: Path with error ellipses using $10\times$ noise – ellipses are far too large.

Now the opposite problem occurs, the bounds (± 500 mm for position, $\pm 15^\circ$ for heading) are far too wide, and the error ellipses in Figure 13 overlap and covers the whole the trajectory.

The two experiments above bracket the real noise. The main learning point is that σ_α is the dominant parameter as it controls heading uncertainty, which then compounds into position drift. After experimenting, we settled on:

Parameter	Tuned value
σ_v	$0.25 \cdot v $
σ_α	2°
σ_T	0.001 s

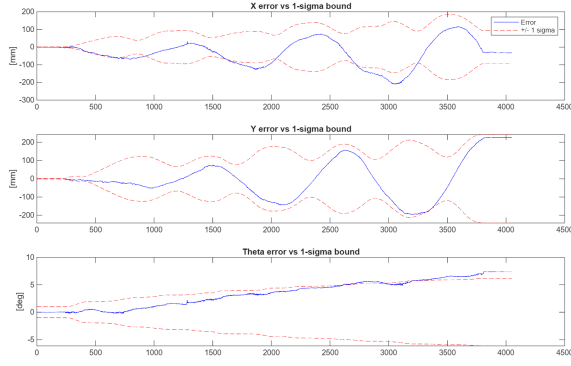


Figure 14: Estimation error (blue) vs predicted $\pm 1\sigma$ bounds (red dashed) with tuned noise parameters.

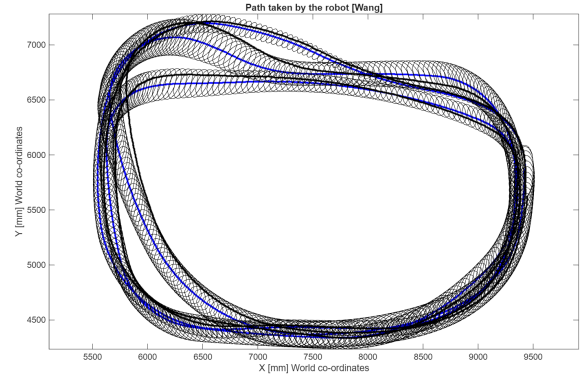


Figure 15: Path with error ellipses using tuned noise parameters.

With these values (Figures 14 and 15), the predicted $\pm 1\sigma$ bounds follow the actual errors much more closely. The errors occasionally touch or slightly exceed the envelope. We learned that that σ_α has the biggest impact as it drives the heading uncertainty, which cascades into position uncertainty at every step. σ_v has a moderate effect on along-track position, and σ_T is negligible.

Task 9: Plot the error ellipses along the path. How long (distance and time) is the path?

Figure 16 shows the error ellipses plotted along the robot's path using the tuned noise parameters from Task 8. Initially the ellipses are small and nearly circular, but they grow larger and more elongated as the run progresses. The orientation of the ellipses rotates with the robot's heading, showing that the uncertainty is correlated with the direction of motion.

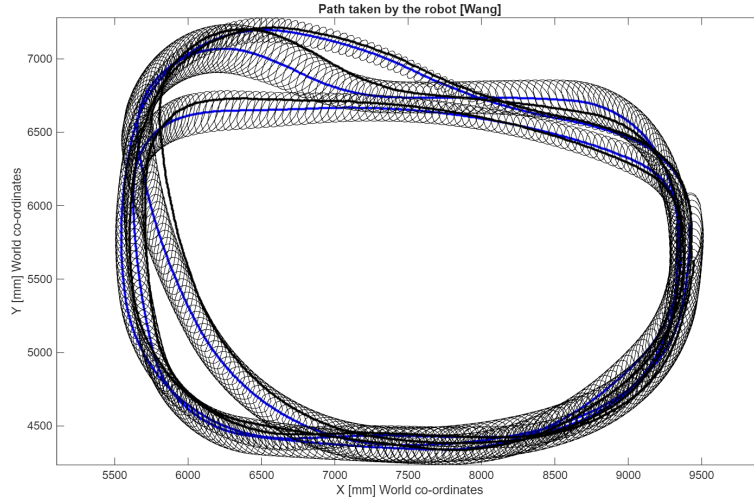


Figure 16: Snowwhite path with error ellipses

To calculate the path length, we sum the forward displacement at each step. The total time is simply the number of samples multiplied by the sampling period:

- **Total Time:** $(N - 1) \times T = (4050 - 1) \times 0.050 = 202.45$ seconds (≈ 3.4 minutes).
- **Total Distance:** Computed by summing $|\Delta d_k| = |v_k \cos(\alpha_k)T|$ over all steps = 31871.65 mm (31.87 meters)

3 Conclusion

In this exercise, we implemented three different kinematic models for odometry and dead-reckoning, and used error propagation to predict how uncertainty evolves during robot motion.

For the differential-drive Khepera robot, we showed that the two odometry models (using $\Delta d, \Delta \theta$ and $\Delta R, \Delta L$ as uncertainty parameters) produce consistent trajectories when correctly calibrated. Wang’s compensation term has negligible effect at high sampling rates but becomes essential at lower rates to preserve the circular arc approximation. We also demonstrated that incorrect wheel diameter and wheelbase parameters cause systematic drift, with the wheelbase error being particularly critical since it affects heading, which compounds into position error over time.

For the steer-drive Snowwhite robot, we derived the Jacobian matrices for the dead-reckoning model with speed, steering angle, and sampling time as uncertainty parameters. The steering angle noise σ_α was found to be the dominant parameter driving heading uncertainty, which cascades into position drift.

Practically, this implies that while dead reckoning provides a smooth short-term estimate, it is insufficient for long-term navigation. An autonomous system needs to fuse this relative estimate with absolute position measurements to collapse the uncertainty ellipses and correct the inevitable drift.

References

- [1] C. M. Wang, “Location Estimation and Uncertainty Analysis for Mobile Robots,” *IEEE International Conference on Robotics and Automation*, 24–29 April 1988, pp. 1231–1235. <https://doi.org/10.1109/ROBOT.1988.12229>
- [2] R. Siegwart and I. R. Nourbakhsh, *Introduction to Autonomous Mobile Robots*. Massachusetts: The MIT Press, 2004.